

Associate Architect 양성과정 · Architecture 설계 · 강사 김덕겸

Day 1 · 과정 실습 소개

Architecture Styles 실습 (Ex1 · Ex2)

- 온라인 영상 학습: Course Introduction · 1.1 ~ 1.7
- Ex1. Data Flow / Data Centered / Implicit Invocation Style 의 QA 영향 분석
- Ex2. Interaction-Oriented / Hierarchical Style 의 QA 영향 분석

시간	1일차
8:30	과정 소개
9:00	
9:30	0. Course Introduction (28min) 1.1 Introduction to Architecture Style (25min) 1.2 Data Flow Architecture (33min) 1.3 Data Centered Architecture (18min) 1.4 Implicit Invocation Architecture (19min)
10:00	
10:30	
11:00	
11:30	
12:00	점심 식사
12:30	실습 소개
13:00	Ex 1. Architecture Style 실습 1 (DFA, DCA, IIA) - 7 styles
13:30	
14:00	1.5 Interaction-Oriented Architecture (31min) 1.6-1 Hierarchical Architecture (25min) 1.6-2 Hierarchical Architecture (26min) 1.7-1 Distributed Architecture (26min)
14:30	
15:00	
15:30	
16:00	Ex 1 발표 및 피드백
16:30	Ex 2. Architecture Style 실습 2 (IOA, HA) - 7 styles
17:00	

Day 1 · 오전

온라인 영상 학습

9:30 ~ 11:30

- 0. Course Introduction (28분)
- 1.1 Introduction to Architecture Style (25분)
- 1.2 Data Flow Architecture (33분)
- 1.3 Data Centered Architecture (18분)
- 1.4 Implicit Invocation Architecture (19분)

※ 영상은 1.2 배속으로 학습하세요.

Lunch Time

11:30 ~ 12:30

시간	1일차
8:30	과정 소개
9:00	
9:30	
10:00	0. Course Introduction (28min) 1.1 Introduction to Architecture Style (25min) 1.2 Data Flow Architecture (33min) 1.3 Data Centered Architecture (18min) 1.4 Implicit Invocation Architecture (19min)
10:30	
11:00	
11:30	
12:00	
12:30	점심 식사
13:00	실습 소개 Ex 1. Architecture Style 실습 1 (DFA, DCA, IIA) - 7 styles
13:30	
14:00	
14:30	1.5 Interaction-Oriented Architecture (31min) 1.6-1 Hierarchical Architecture (25min) 1.6-2 Hierarchical Architecture (26min) 1.7-1 Distributed Architecture (26min)
15:00	
15:30	
16:00	
16:30	Ex 1 발표 및 피드백
17:00	Ex 2. Architecture Style 실습 2 (IOA, HA) - 7 styles

Day 1 · 오후

Ex1 실습 시간

12:30 ~ 14:00

- Data Flow · Data Centered · Implicit Invocation Style 의 품질속성 영향 분석 (조별)
- 실습 템플릿: ex1-3_Architecture style exercise-template.xlsx

Ex 1 ~ 3

Architecture Styles 실습 소개

Architecture Style 이 품질속성(QA)에 미치는 영향을 분석하고 근거를 제시한다

- 실습 개요 · 시간표 · 환경과 결과물
- 품질속성 영향도 분석 방법
- 실습 관련 이론: Quality Attribute · QA Scenario · Style 별 특징

Architecture Styles 실습 시간표

실습 활동	시간 (분)
Architecture Styles 실습 소개	30
Ex1. Architecture Styles 실습 1 (Data Flow · Data Centered · Implicit Invocation)	
조별 실습	60
모범답안 설명 & 조별 실습 결과 리뷰	30
Ex2. Architecture Styles 실습 2 (Interaction-Oriented · Hierarchical)	
조별 실습	60
모범답안 설명 & 조별 실습 결과 리뷰	30
Ex3. Architecture Styles 실습 3 (Distributed · Deployment)	
조별 실습	60
모범답안 설명 & 조별 실습 결과 리뷰	30

Architecture Styles 실습 환경과 결과물

■ 실습 환경 및 도구

- 실습 템플릿: **ex1-3_Architecture style exercise-template.xlsx** (Excel 파일)

■ 실습 결과물

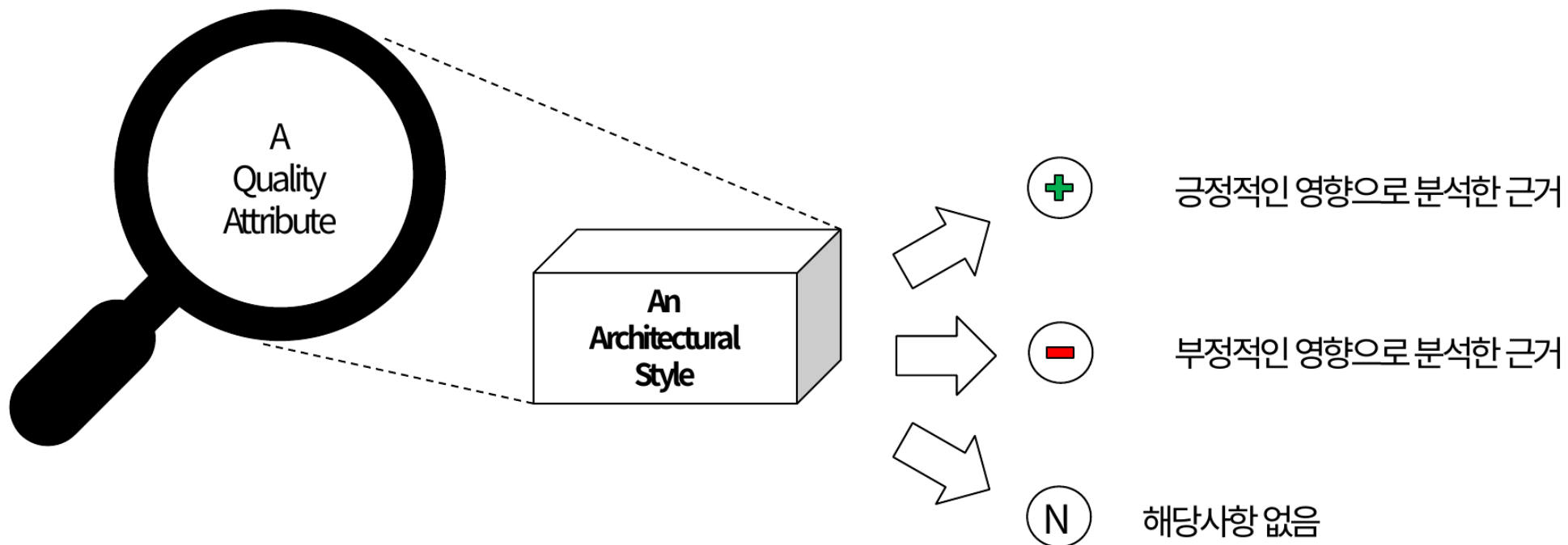
- 조별 실습 결과물은 Café 에 공지된 링크의 조별 Sheet 에 실습 내용을 복사하여 붙여넣기

■ 실습 Hints

- Effect(++ , +, 0, -, --) 로 명기한 값 자체는 중요하지 않다. 적용 대상과 평가자에 따라 달라질 수 있는 값이다
- 단, **Justification** 에 기술한 내용의 논리적 타당성과, 그 내용이 **Style** 의 정의와 모순되지 않는지를 확인한다

품질속성 영향도 분석 방법 (1)

- 하나의 **Architecture Style** 을 하나의 **Quality Attribute** 의 렌즈로 들여다보고, 영향의 방향과 그 근거를 적는다



품질속성 영향도 분석 방법 (2)

■ 품질속성에 대한 영향도를 5단계로 평가한다: ++ + 0 - --

Data Flow	Batch Sequential	
	Effects	Justification
Availability	--	프로세싱 스텝 중 하나라도 문제가 생기면 전체 시스템이 문제가 생긴다.

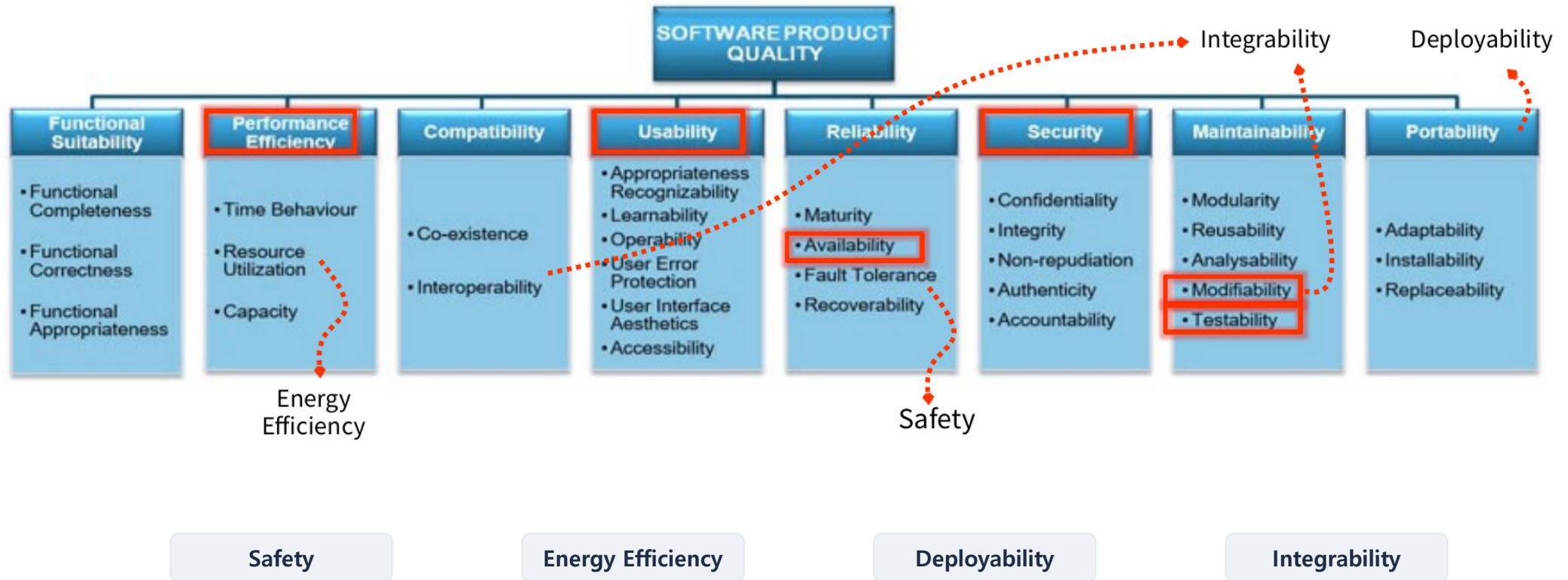
■ 하나의 Style 에 대한 영향도는 관점에 따라 여러 값으로 평가될 수 있다

- 예: Shared Repository Style 의 Modifiability
 - + 데이터 Client 의 변경이 다른 Client 에 영향을 주지 않는다
 - Data Store 의 자료구조 변경은 의존하는 모든 Client 에 영향을 준다
- 다만 실습 스프레드시트에서는 전반적인 영향도를 고려하여 **하나의 값**으로 평가한다

Implicit Invocation	Buffered Message Based	
	Effects	Justification
Availability	+	+ 하나의 컴포넌트 오류가 전체 시스템에 큰 영향을 주지 않는다. - Message Buffer의 가용성은 전체 시스템의 가용성에 중요한 영향을 준다.

Quality Attributes — 품질 모델 (ISO/IEC 9126 계열)

- 제품 품질 모델의 특성 중 이번 실습에서 다루는 10개 품질속성 (붉은 테두리) 과 SAiP(Bass) 에서 추가된 4개 속성



Safety · Energy Efficiency · Deployability · Integrability 는 9126 모델에는 없으나 SAiP 4판이 별도 품질속성으로 다룬다

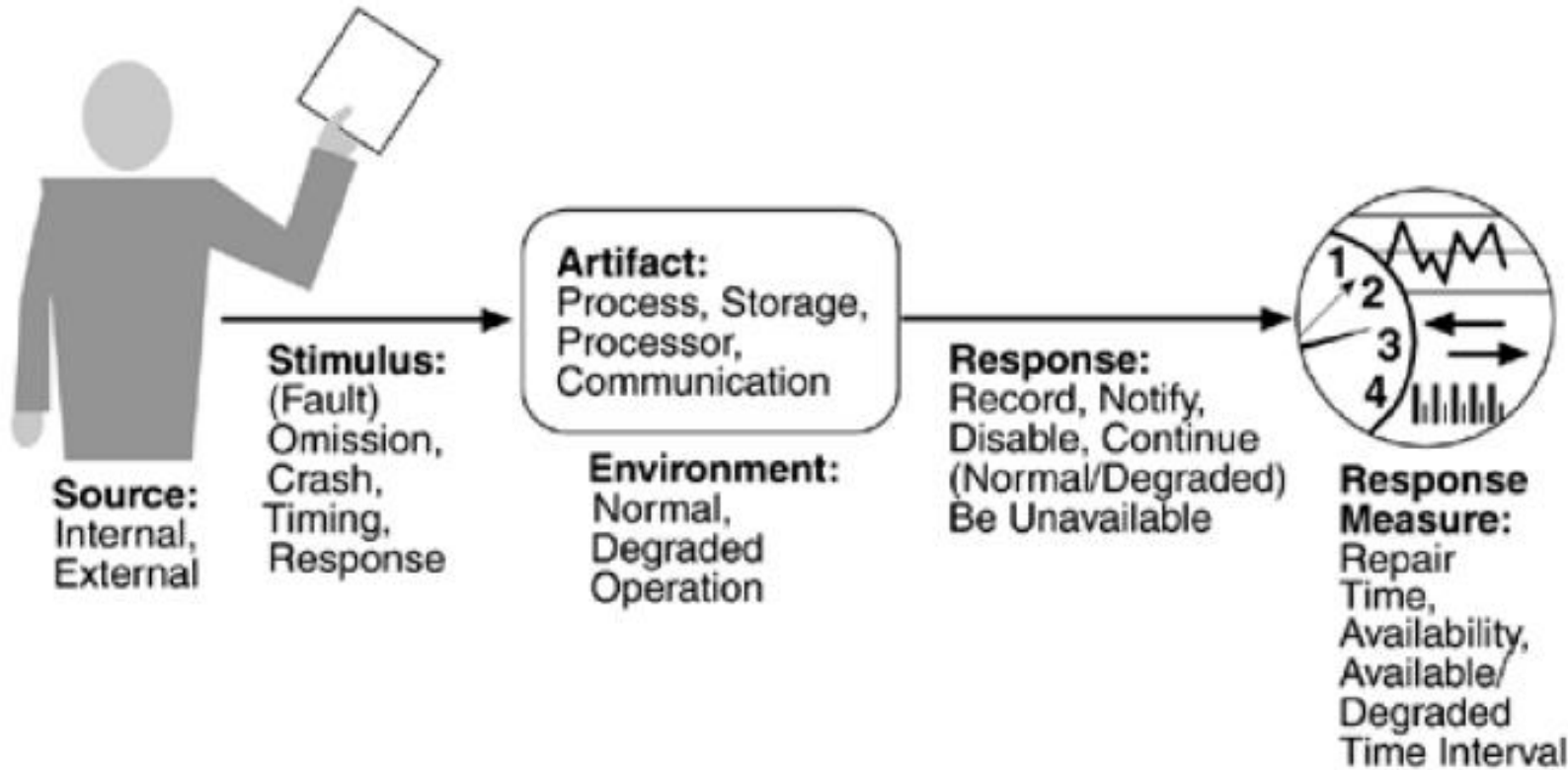
Quality Attributes — 10개 품질속성의 정의

품질 속성	내용
Availability (가용성)	언제든지 요청될 때 시스템이 지정된 동작 가능 상태에 있는 정도. 또는 Fault 발생 시 정상 상태로 복구할 수 있는 능력
Modifiability (변경 용이성)	시스템을 쉽게 변경할 수 있는 정도와, 시스템이 변경에 적응하는 유연성
Performance (성능)	이벤트가 발생할 때 시스템이 특정 작업을 수행하는 데 걸리는 시간(Time Behavior), 사용하는 자원(Resource Utilization, CPU 점유율 등)
Security (보안성)	악의적인 행동의 가능성과 정보의 도난 또는 손실 가능성을 줄이는 시스템의 능력
Safety (안전성)	시스템의 잘못된 동작이나 장애로 인해 인명, 재산, 환경 등에 피해가 발생하지 않도록 하는 능력 (= Fault Tolerance)
Testability (시험 용이성)	소프트웨어가 테스트를 통해 결함을 드러낼 수 있는 용이성
Deployability (배포 용이성)	소프트웨어가 예측 가능하고 허용 가능한 시간과 비용 내에서 운영 환경에 배포될 수 있는 능력 (= Portability)
Usability (사용성)	사용자(End User)가 원하는 작업을 수행하는 것이 얼마나 쉬운지
Energy Efficiency (에너지 효율성)	주어진 기능을 수행하는 동안 소비되는 에너지의 양을 최소화할 수 있는 능력 (= Performance Efficiency · Resource Utilization)
Integrability (통합 용이성)	소프트웨어 컴포넌트나 시스템이 다른 시스템·컴포넌트와 쉽게 통합될 수 있는 능력 (= Modifiability + Interoperability)

SOT: System On Time

Quality Attribute Scenario — 6 요소

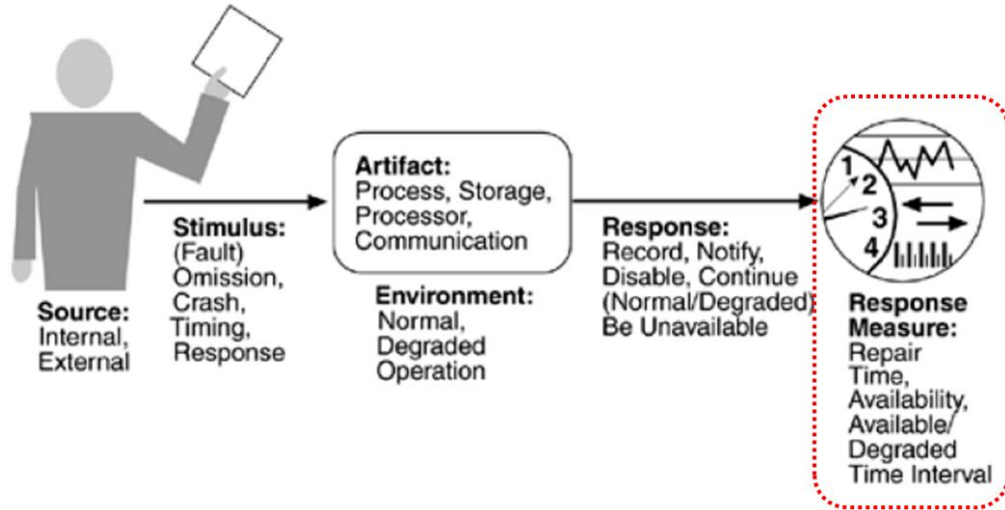
- QA 요구는 **Source · Stimulus · Artifact · Environment · Response · Response Measure** 의 6 요소로 기술한다



반드시 'Measurable Value' 가 선정되어야 한다!

- Response Measure 가 없으면 QA 는 검증할 수 없는 희망 사항이 된다
- 실습 평가의 절대 기준

Quality Attribute Scenario — Measure 의 의도



Availability 시나리오 예

- Source : component
- Stimulus : occur exception
- Artifact : system
- Response : performs recovery actions
- Measure : **within 1 sec**



■ "within 1 sec" 는 Availability 인가, Performance(Time Behavior) 인가?

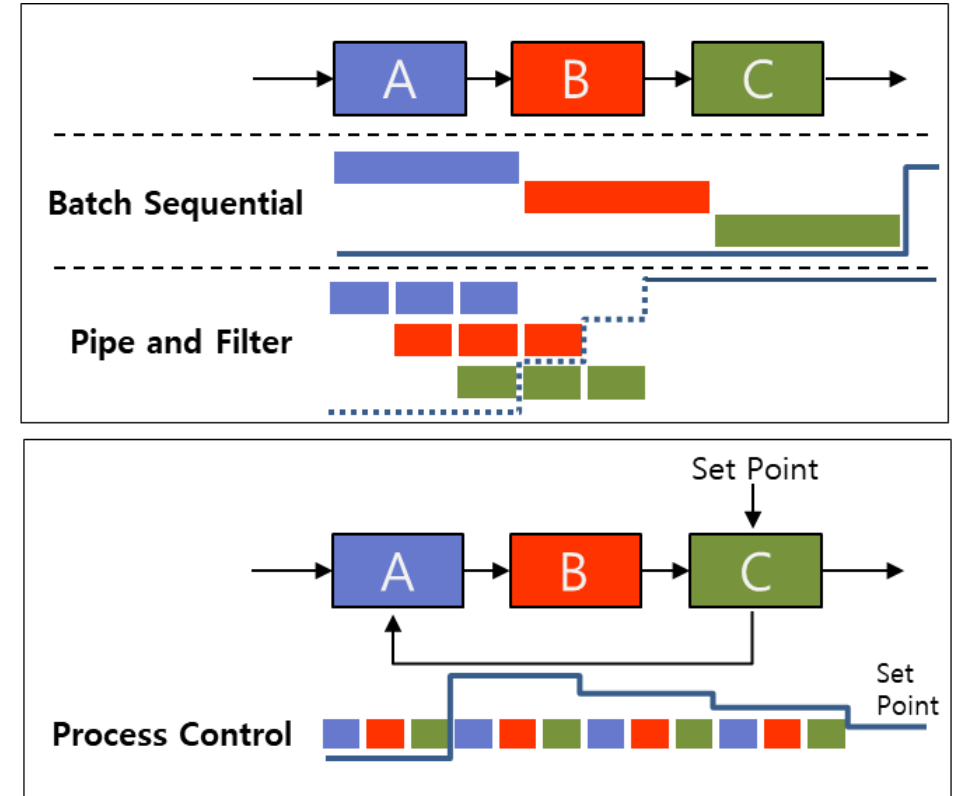
- 측정값의 단위(시간)가 아니라 **측정의 의도**를 생각하라 — 복구 능력을 재려는 것이면 Availability 이다
- 이렇게 하지 않으면 QA 가 '측정 가능한 단위' 로 잘못 제한되어 버린다

Data Flow Architecture

- 연속적으로 입력된 데이터를 일련의 변환 과정을 통해 처리하는 구조
- 데이터 흐름은 명확하지만 다른 상호작용은 잘 보이지 않는다

■ 3 가지 Style

- **Batch Sequential** 기능 분리 + 순차성 + 정보 집중 + 강한 결합
 - Run to completion — 앞 단계가 끝나야 다음 단계 시작
- **Pipe and Filter** 역할 분리 + 동시성 + 정보 분산 + 약한 결합
 - Concurrent and incremental transformation of streams
- **Process Control** 기능 분리 + 순차성 + 강한 결합
 - Looping, Set Point — 피드백 루프로 의미적·시간적 결합이 강해진다



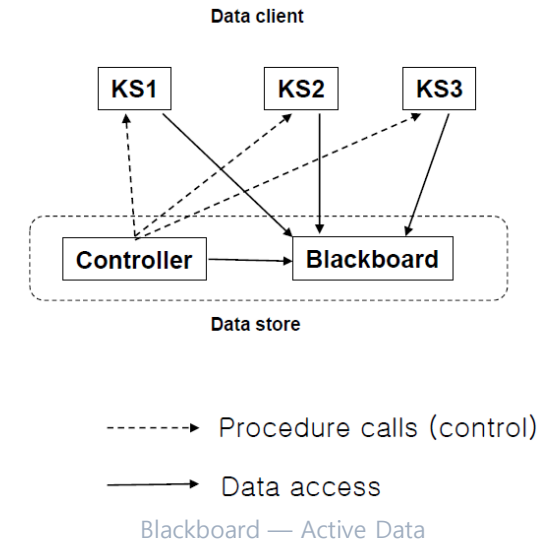
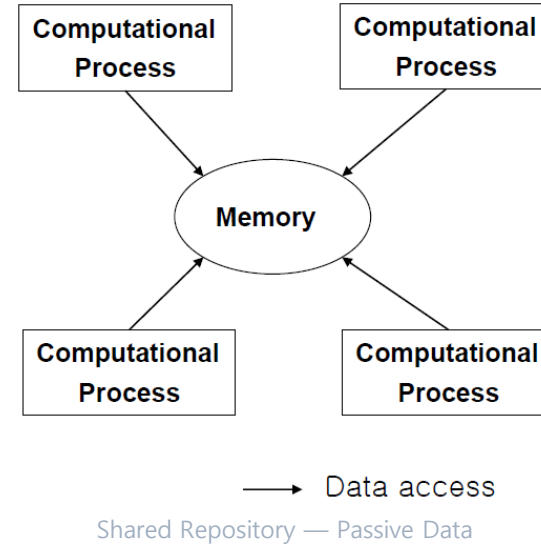
위: Batch Sequential / Pipe and Filter / Process Control

Data Centered Architecture

- 데이터 중심의 상호작용을 처리하는 구조
- 데이터 상호작용은 명확하지만 흐름은 잘 보이지 않는다

■ 2 가지 Style

- **Shared Repository** 역할 분리 + 동시성 + 정보 집중 + 약한 결합
 - Data Store = Passive / Clients = Active
- **Blackboard** 역할 분리 + 동시성 + 정보 집중 + 약한 결합
 - Data Store = Active / Clients (Knowledge Source) = Passive
 - 구성 요소 간 Dependency (Controller ↔ KS) 가 발생 → 그 영향을 생각해 볼 것



Implicit Invocation Architecture

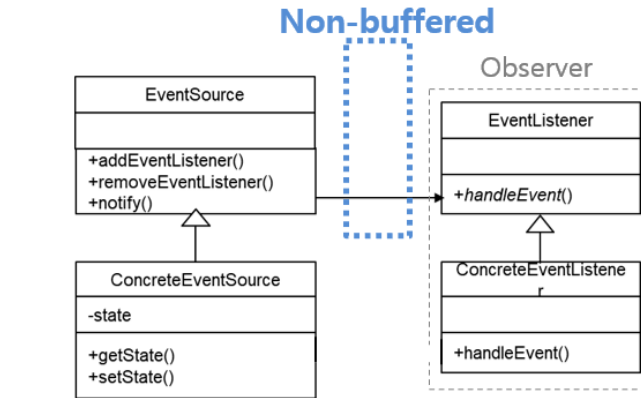
- 발신자와 수신자가 서로 모르고 통신한다 (약한 결합)
- 직접 호출이 아니다 — 이벤트 발생에 관심을 등록한 프로시저가 호출된다

■ Non-buffered Event-based Style

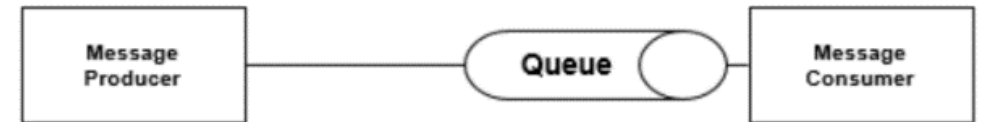
- 기능 분리 + 순차성(Synchronous) + 강한 결합
- 이벤트 소스가 리스너를 같은 프로세스-스레드에서 직접 호출

■ Buffered Message-based Style

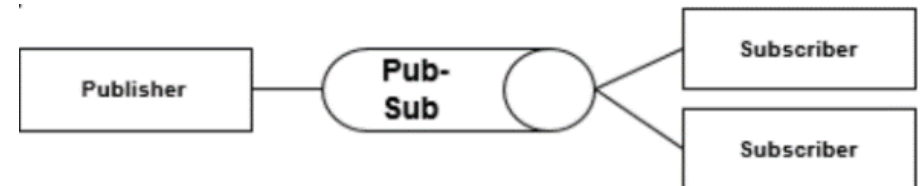
- 역할 분리 + 동시성(Asynchronous) + 약한 결합
- Point-to-Point Message Architecture (one-to-one)
- Publish-Subscribe Message Architecture (one-to-many)



Non-buffered (Observer)



P2P Messaging Architecture



Pub-Sub Messaging Architecture

시간	1일차
8:30	과정 소개
9:00	
9:30	0. Course Introduction (28min) 1.1 Introduction to Architecture Style (25min) 1.2 Data Flow Architecture (33min) 1.3 Data Centered Architecture (18min) 1.4 Implicit Invocation Architecture (19min)
10:00	
10:30	
11:00	
11:30	
12:00	점심 식사
12:30	실습 소개
13:00	Ex 1. Architecture Style 실습 1 (DFA, DCA, IIA) - 7 styles
13:30	
14:00	1.5 Interaction-Oriented Architecture (31min) 1.6-1 Hierarchical Architecture (25min) 1.6-2 Hierarchical Architecture (26min) 1.7-1 Distributed Architecture (26min)
14:30	
15:00	
15:30	
16:00	
16:30	Ex 1 발표 및 피드백
17:00	Ex 2. Architecture Style 실습 2 (IOA, HA) - 7 styles

Day 1 · 오후

온라인 영상 학습

14:00 ~ 16:00

- 1.5 Interaction-Oriented Architecture (31분)
- 1.6-1 Hierarchical Architecture (25분)
- 1.6-2 Hierarchical Architecture (26분)
- 1.7-1 Distributed Architecture (26분)

※ 영상은 1.2 배속으로 학습하세요.

시간	1일차
8:30	과정 소개
9:00	
9:30	0. Course Introduction (28min) 1.1 Introduction to Architecture Style (25min) 1.2 Data Flow Architecture (33min) 1.3 Data Centered Architecture (18min) 1.4 Implicit Invocation Architecture (19min)
10:00	
10:30	
11:00	
11:30	
12:00	점심 식사
12:30	실습 소개
13:00	Ex 1. Architecture Style 실습 1 (DFA, DCA, IIA) - 7 styles
13:30	
14:00	1.5 Interaction-Oriented Architecture (31min) 1.6-1 Hierarchical Architecture (25min) 1.6-2 Hierarchical Architecture (26min) 1.7-1 Distributed Architecture (26min)
14:30	
15:00	
15:30	
16:00	Ex 1 발표 및 피드백
16:30	Ex 2. Architecture Style 실습 2 (IOA, HA) - 7 styles
17:00	

Day 1 · 오후

Ex2 실습 시간

16:00 ~ 17:30

- Interaction-Oriented · Hierarchical Style 의 품질속성 영향 분석 (조별)

Interaction-Oriented Architecture

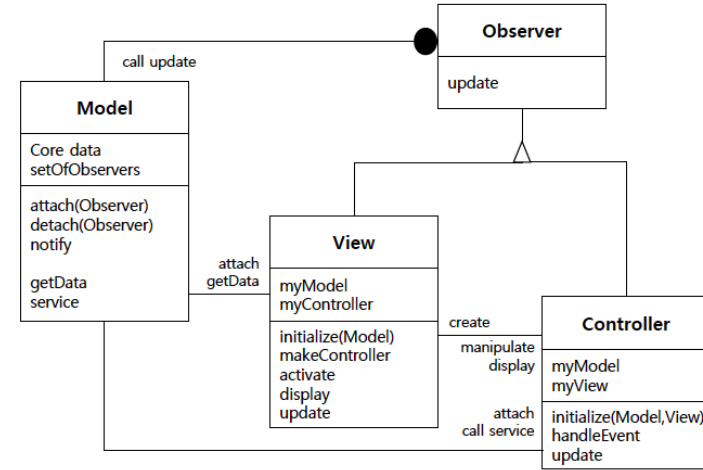
- 사용자와 구성 요소 간의 상호작용에 중점을 둔 구조

■ Model-View-Controller (MVC)

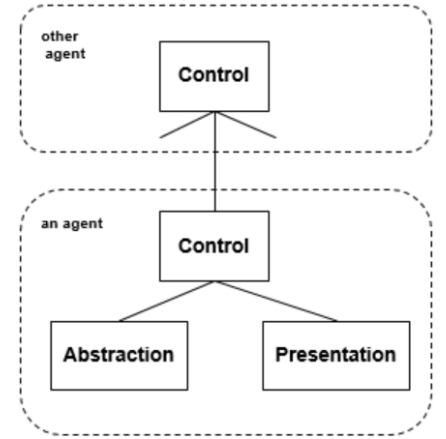
- 기능 분리 + 정보 집중 (+ 약한/강한 결합 혼재)
- 하나의 Model 에 여러 개의 View 가 쉽게 교체되는 상황
- 정보의 집중 발생 / Model-View 간의 의존성

■ Presentation-Abstraction-Control (PAC)

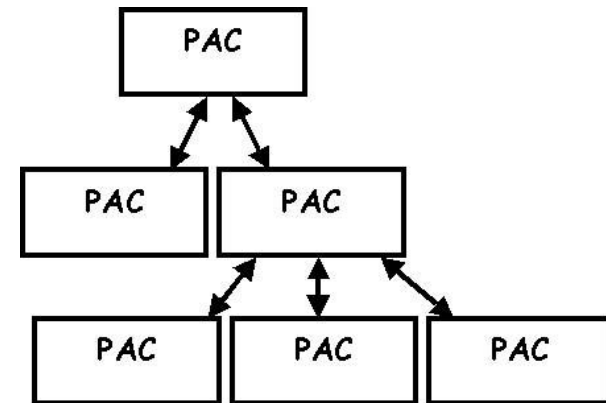
- 기능 분리 + 동시성 + 정보 분산 + 약한 결합
- Agent 기반의 계층(hierarchical) 구조 — 주로 복잡한 GUI 시스템 (예: Web Application)
- 정보 저장 위치의 분산 (각 Agent 의 Abstraction)



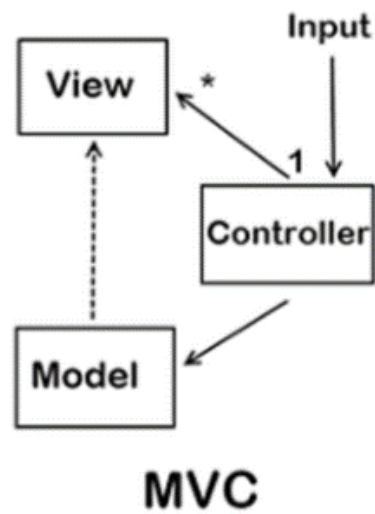
MVC Style



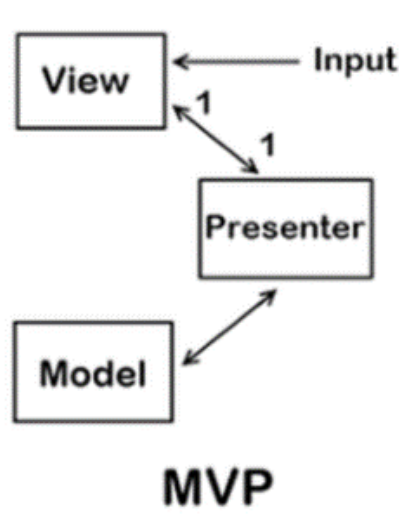
PAC Style



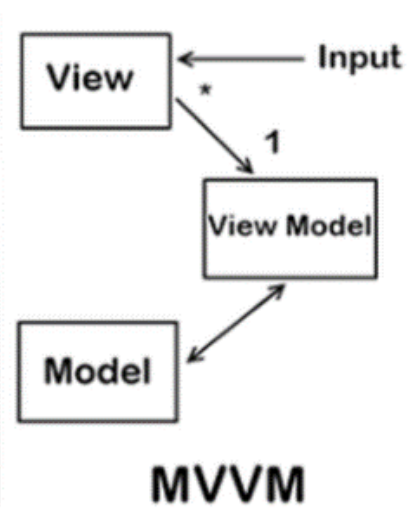
MV* Style — MVC · MVP · MVVM



Web (MPA)



Web (SPA: Angular / React)



WPF · Web (MPA: Vue.js)

■ Model-View-Presenter (MVP)

- Pros: View-Model 간 의존성 제거
- Cons: View-Presenter 간 의존성 ↑ (View-Presenter 1:1 관계)

■ Model-View-ViewModel (MVVM)

- Pros: View-Model 간 의존성 제거 (Command 패턴, Data Binding 이용)
- Cons: ViewModel 설계 난이도 ↑

MVC Style

■ 1. 구조

- Model: 애플리케이션에서 사용되는 데이터와 그 데이터를 처리하는 부분
- View: 사용자에게 보여지는 UI 부분
- Controller: 사용자의 입력(Action)을 받고 처리하는 부분

■ 2. 동작

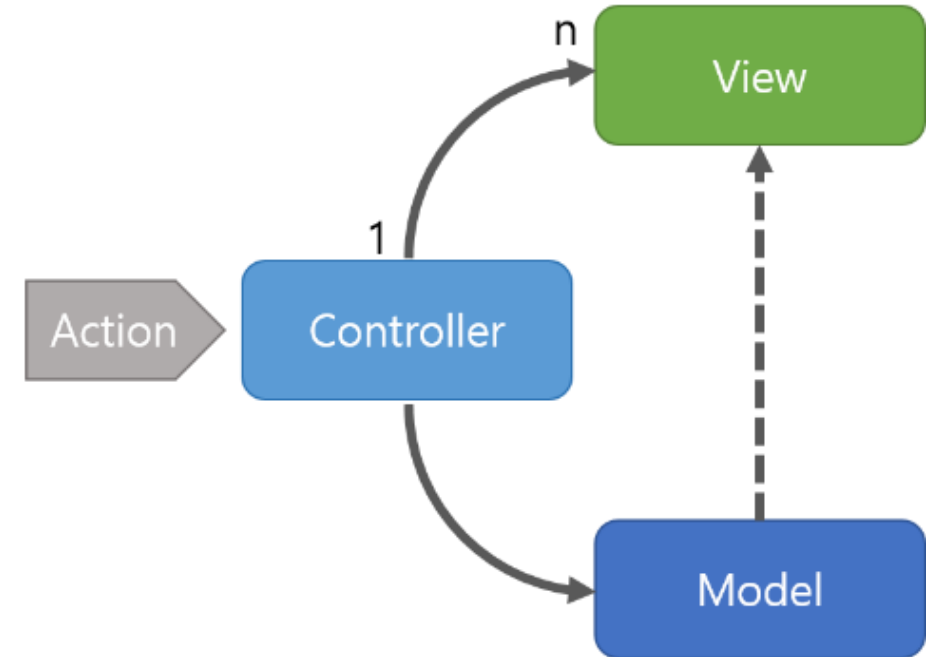
- 사용자의 Action 이 Controller 에 들어온다 → Controller 는 Action 을 확인하고 Model 을 업데이트한다
- Controller 는 Model 을 나타내 줄 View 를 선택한다 → View 는 Model 을 이용하여 화면을 나타낸다

■ 3. 특징

- Controller 는 여러 개의 View 를 선택할 수 있는 1:n 구조
- Controller 는 View 를 선택할 뿐 직접 업데이트하지 않는다 (View 는 Controller 를 알지 못한다)

■ 4. 장·단점

- 장점: 가장 단순하고 널리 사용되는 패턴
- 단점: View 와 Model 사이의 의존성이 높다 → 애플리케이션이 커질수록 복잡해지고 유지보수가 어려워진다



참조: <https://beomy.tistory.com/43>

MVP Style

■ 1. 구조

- Model: 애플리케이션에서 사용되는 데이터와 그 데이터를 처리하는 부분
- View: 사용자에게 보여지는 UI 부분
- Presenter: View 에서 요청한 정보로 Model 을 가공하여 View 에 전달해 주는 부분

■ 2. 동작

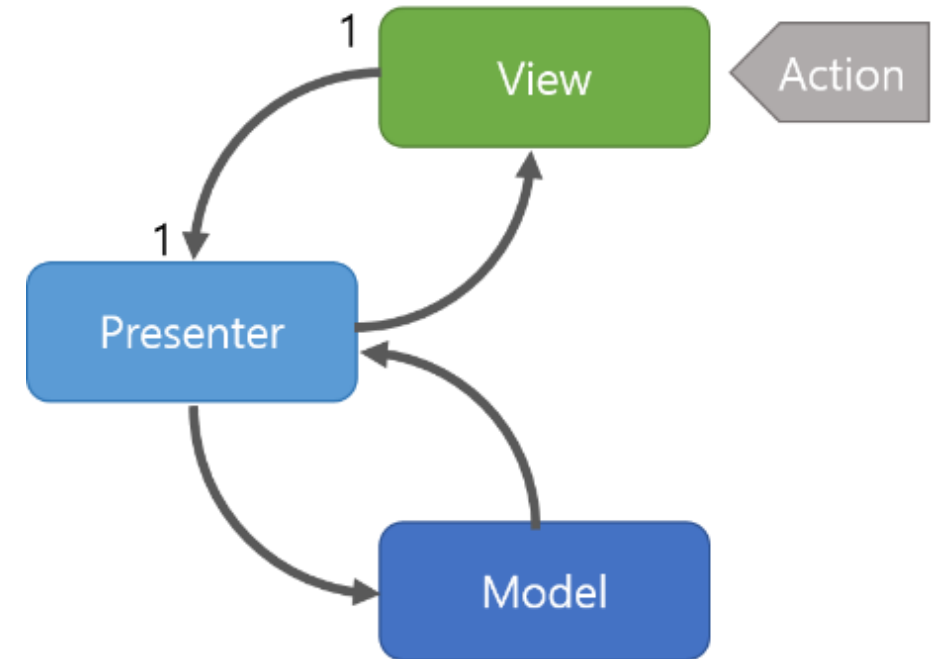
- 사용자의 Action 이 View 를 통해 들어온다 → View 는 데이터를 Presenter 에 요청한다
- Presenter 는 Model 에 데이터를 요청하고, Model 은 응답한다 → Presenter 는 View 에 데이터를 응답한다
- View 는 Presenter 가 응답한 데이터를 이용하여 화면을 나타낸다

■ 3. 특징

- Presenter 는 View 와 Model 의 인스턴스를 가지고 둘을 연결하는 접착제 역할
- Presenter 와 View 는 1:1 관계

■ 4. 장·단점

- 장점: View 와 Model 의 의존성이 없다 (Presenter 를 통해서만 데이터를 전달)
- 단점: View 와 Presenter 사이의 의존성이 높아진다 → 애플리케이션이 커질수록 강한 의존성이 생긴다



참조: <https://beomy.tistory.com/43>

MVVM Style

■ 1. 구조

- Model: 애플리케이션에서 사용되는 데이터와 그 데이터를 처리하는 부분
- View: 사용자에게 보여지는 UI 부분
- View Model: View 를 표현하기 위해 만든 View 를 위한 Model — View 를 나타내기 위한 데이터 처리를 담당

■ 2. 동작

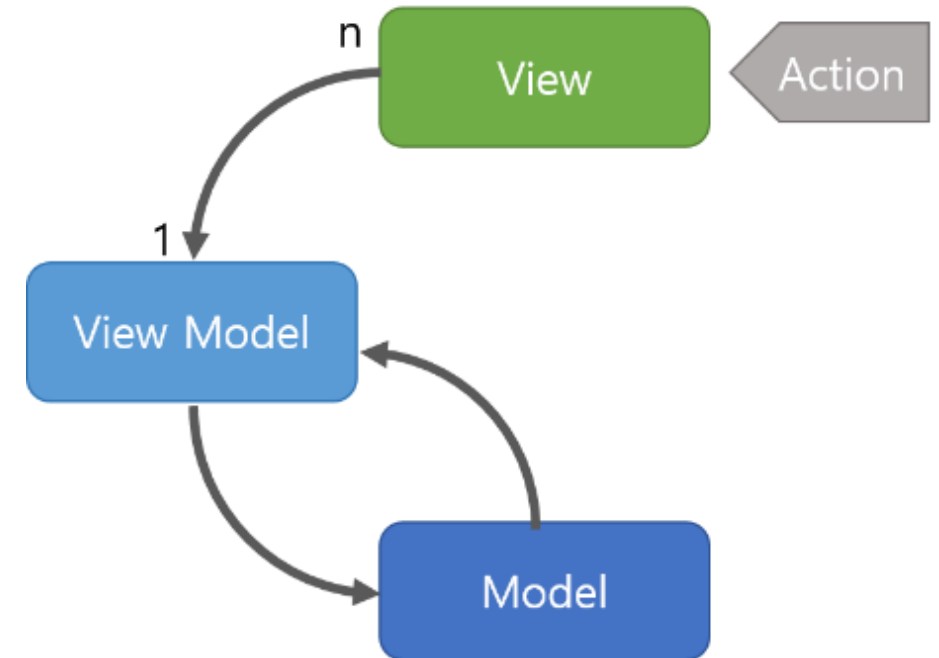
- 사용자의 Action 이 View 를 통해 들어온다 → View 는 Command 패턴으로 View Model 에 Action 을 전달한다
- View Model 은 Model 에 데이터를 요청하고, 응답받은 데이터를 가공하여 저장한다
- View 는 View Model 과 Data Binding 하여 화면을 나타낸다

■ 3. 특징

- Command 패턴과 Data Binding 두 가지를 사용하여 구현 (Command 패턴이 필수는 아님)
- Command 패턴과 Data Binding 으로 View 와 View Model 사이의 상호 의존성을 없앴다
- View Model 과 View 는 1:n 관계

■ 4. 장·단점

- 장점: View 와 Model 사이의 의존성이 없고, View 와 View Model 사이의 의존성도 없다 → 각 부분이 독립적이라 모듈화하여 개발할 수 있다
- 단점: View Model 의 설계가 쉽지 않다



참조: <https://beomy.tistory.com/43>

Hierarchical Architecture

■ Main-Subroutine

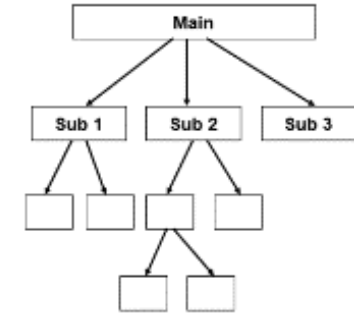
- 기능 분리 + 순차성 + 정보 집중 + 강한 결합
- 강한 결합 구조 (Call-Return), 단일 Thread/Process

■ Master-Slave

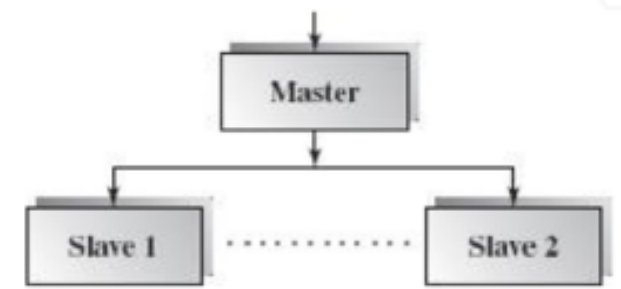
- 역할 분리 + 동시성 + 정보 분산 (+ 약한 결합)
- 느슨한 결합 (Service Request/Response)
- Slave 는 복제된 다수 서비스(같은 기능·다른 알고리즘, 또는 다른 기능)를 제공

■ Layer Based (Layered · Virtual Machine · Micro-Kernel: Plug-In)

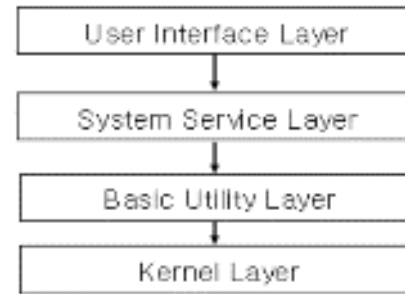
- 가장 거시적·추상적·일반적인 Architecture
- 공통 장점: 독립성, 보안성, 재사용성
- 공통 단점: Layer 가 많으면 성능 저하, 하위 Layer 의존성



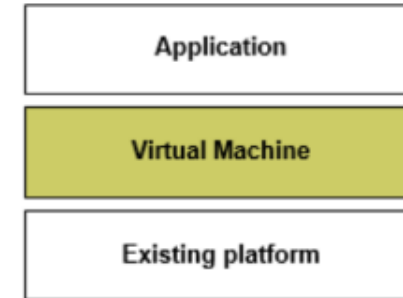
Main-Subroutine ← Call-Return



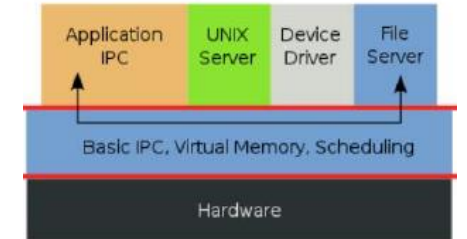
Master-Slave ← Service (Request/Response)



Layered



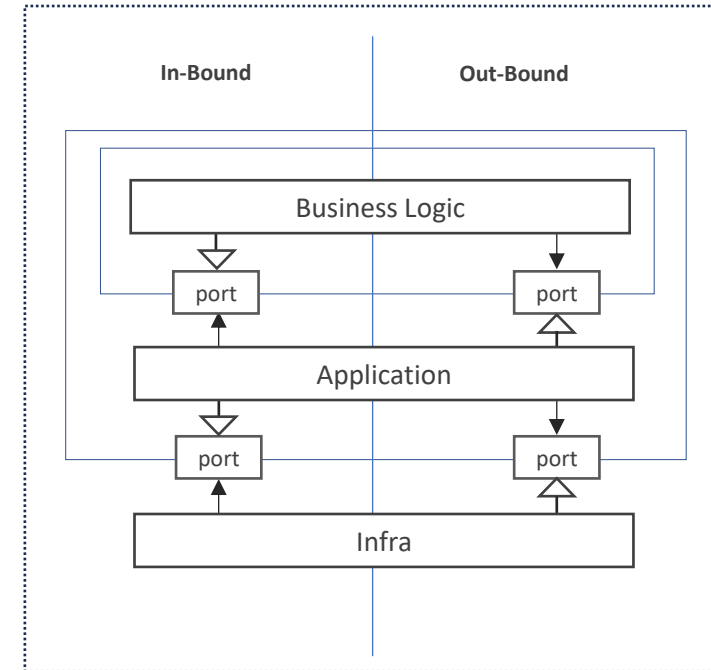
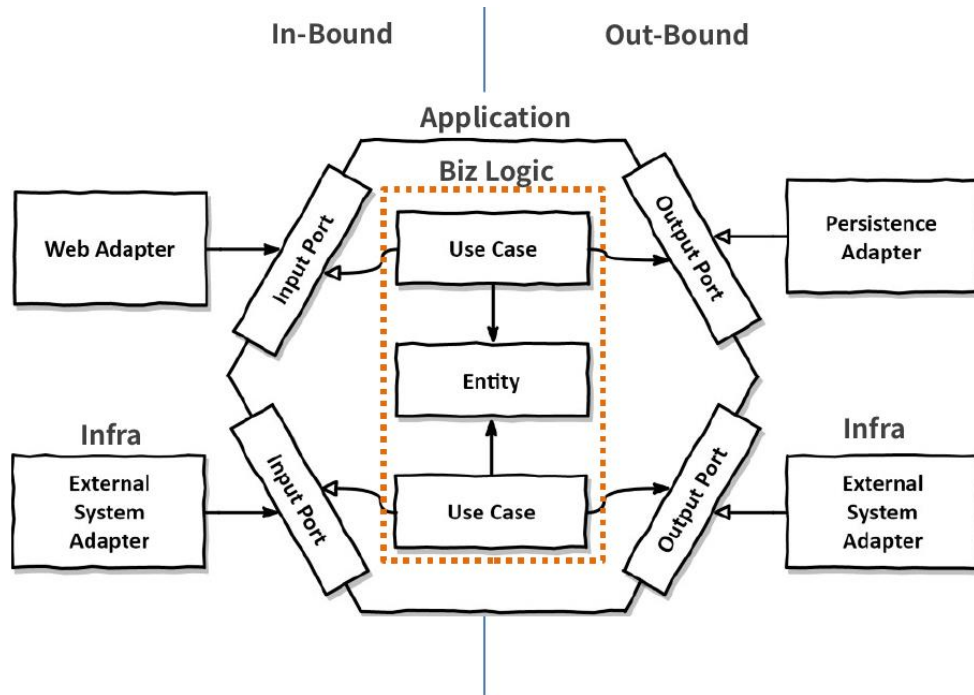
Virtual Machine / Container
역할 분리 + 정보 집중 + 약한 결합 → Portability



Micro-Kernel (Plug-In)
기능/역할 분리 + 정보 집중 + 약한 결합 → Extensibility

Hexagonal Architecture (Ports & Adapters)

- 하위 Layer 의존성을 극복하기 위한 구조 — Application(Biz Logic) 이 중심에 있고, In-bound / Out-bound **Port** 를 통해 Infra(Adapter) 와 연결된다



Hexagonal as Layered Architecture — Layered 의 하위 의존 방향을 Port 로 뒤집는다